

in the definition of the function. Normally this is supposed to be the last statement of the function, else the C++ compiler will generate errors or a warning shall be issued in the absence of a return statement.

5.2 Function Prototyping

This is one of the most important features in C++. The prototype mainly ensures that whenever a function is called, it is used properly with its right parameters. It also describes the function interface to the compiler by giving details about the type of arguments and return values. This helps to conveniently call the function rather than the entire definition. It is usually defined as a declaration statement in the calling program. The main form of declaring a function prototype is as follows:

```
Return type name (argument_type1, argument_type2, ...);
```

Return type refers to the data type returned by the function. name is related to the name of the function, while argument_type1, argument_type2, etc. refer to parameters passed to the function when it is called.

Consider the following example:

```
float volume (int m, float n, float o);
```

Here, volume is the name of the function that returns a float value. Parameters that are passed within this float function include m(int), n(float) and o(float). The prototype described above is identical to a function definition, except that this is not related to the function body. The parameters that are passed and enumerated don't need to include identifiers, but only the type specifiers are included. In the prototype declaration, it is optional to include a name for each parameter. For instance let us declare a function called Example which accepts two parameters of type integer. This is shown below: